

Claude Project Sync

Product Specification v5

Generated February 03, 2026

github.com/christreadaway/claude-project-sync

Author	Chris Treadaway
Version	v5 (February 2026)
Status	Active / In Production
Repository	christreadaway/claude-project-sync
License	Open Source

Table of Contents

1. Product Overview & Problem Statement
2. User Story
3. Core Functionality & Workflow
4. Inputs & Outputs
5. Business Rules & Logic
6. Data Requirements
7. Integrations & Dependencies
8. Dashboard Visual Specification
9. Current Project Statuses
10. Shared Assets Registry
11. Out of Scope
12. Open Design Questions
13. Success Criteria
14. Future Enhancements

1. Product Overview & Problem Statement

Product Name: Claude Project Sync

Repository: github.com/christreadaway/claude-project-sync

Claude Project Sync solves a fundamental problem for anyone managing multiple projects across Claude Code (terminal) and Claude.ai (web/mobile chat): **these tools don't share memory**. Every time you switch between them, you lose context. You repeat yourself. You forget what was built, what was decided, and where things stand.

The Three Core Problems

Context Amnesia. Claude Code and Claude.ai operate in complete isolation. A full day of work in Claude Code is invisible to Claude.ai, and vice versa. You end up re-explaining project state at the start of every session.

Asset Amnesia. Reusable components built in one project (auth flows, notification systems, UI patterns) are forgotten when you start another project. You rebuild things you've already built.

Status Blindness. With a dozen active projects, there's no single view of where everything stands. Progress lives in scattered conversations and terminal sessions with no dashboard or summary.

Claude Project Sync bridges the gap with local status files, automated PDF reports, and a shared asset registry. The friction to keep things in sync is measured in seconds, not minutes.

2. User Story

Primary Persona

A technology leader or product builder managing multiple simultaneous software projects who uses both Claude Code (for terminal-based development) and Claude.ai (for strategy, writing, and chat-based iteration). They are not necessarily a developer themselves, but they build product and understand business requirements deeply.

User Stories

- As a multi-project builder, I want Claude.ai to know what I just built in Claude Code so I don't have to re-explain every session.
- As a product manager, I want a daily visual dashboard showing where all my projects stand so I can prioritize my time.
- As someone who builds tools for communities (schools, churches), I want to know if I've already built a component in one project before rebuilding it in another.
- As a non-developer, I want the sync process to be nearly automatic, requiring no more than 5 seconds of manual effort per session.

3. Core Functionality & Workflow

System Architecture

The system works within a key constraint: Claude Code can read/write local files but cannot access Google Docs. Claude.ai can read uploads and Google Docs but cannot write to local files. The bridge is a local markdown file (PROJECT_STATUS.md) that both tools can interact with, plus an automated PDF generator that creates a visual dashboard.

Capability	Claude Code	Claude.ai
Read local files	Yes	Upload or paste
Write local files	Yes	No (download only)
Read Google Docs	No	Yes (read only)
Write Google Docs	No	No

Daily Workflow

When	What Happens	Who / What Does It
6:00 AM	PDF dashboard auto-generates with charts showing all project progress	Automation (Magress)
During work	Claude Code updates PROJECT_STATUS.md at end of each session	Claude Code (automatic)
Switching to chat	User uploads PDF or pastes status file to Claude.ai	User (~5 seconds)
During chat	Claude.ai generates updated status; user downloads and saves	Claude.ai (~5 seconds)
End of day	Status files reflect full day of work across both tools	Automatic + minimal manual

Key Files

File	Location	Purpose
PROJECT_STATUS.md	Each project root folder	Running journal of progress per project
generate_status_pdf.py	~/claude-project-sync/	Creates daily visual PDF report with charts
init_project_status.py	~/claude-project-sync/	Initializes status files for new projects
sync_history.py	~/claude-project-sync/	Imports Claude Code session history + builds asset registry
daily_status_report.sh	~/claude-project-sync/	Shell wrapper for 6am automation
CLAUDE.md	~/.claude/	Global instructions for Claude Code (sync reminders, auto-save)
com.ctreada.dailystatusreport.plist	~/Library/LaunchAgents/	Mac launchd config for 6am scheduling

4. Inputs & Outputs

What Goes In

Input	Source	Format	How It Gets There
Session progress	Claude Code	Markdown appended to PROJECT_STATUS.md	Automatic at session end
Project metadata	User / init script	YAML-like header in PROJECT_STATUS.md	One-time setup per project
Manual status updates	Claude.ai	Text in chat	User pastes or describes progress
Reusable asset flags	Claude Code	Tagged entries in PROJECT_STATUS.md	Claude Code logs when prompted

What Comes Out

Output	Destination	Format	Frequency
Daily PDF dashboard	User's Mac (~/claude-project-sync/)	PDF with charts + tables	Daily at 6am
Updated PROJECT_STATUS.md	User's project folder	Markdown	Every Claude Code session
SHARED_ASSETS.md	~/claude-project-sync/	Markdown registry	On demand (--assets flag)
Clipboard-ready status	User's clipboard	Plain text	On demand (cat pbcopy)

What the User Sees

The primary artifact the user interacts with is the **daily PDF dashboard** (detailed in Section 8). It provides a single-page visual overview of all projects: horizontal progress bars ranked by completion, a pie chart of GitHub repo status, a bar chart of projects by category, and two tables splitting projects by whether they have a GitHub repo. Non-project items (content, operations, research) are listed thematically below the charts.

5. Business Rules & Logic

Status file naming: Each project gets PROJECT_STATUS_[ProjectName].md. The filename must match the project name used in the PDF generator's data array.

Progress percentages: Manually set in the PROJECTS data dictionary inside generate_status_pdf.py. Range: 0-100. Rounded to nearest 5% for visual clarity.

GitHub repo flag: Projects are split into two groups: 'with_repos' and 'without_repos'. Projects without repos get a warning flag and an 'Action Needed' column.

Category assignment: Every project is assigned exactly one category: Infrastructure, School, Church, Product, Research, or Personal. Categories drive the bar chart grouping.

Progress log format: Each entry must include: date, 'What was built', 'What was figured out', 'Still stuck on', and 'Next time'. Entries are written in natural language, not jargon.

Sync timestamp: PROJECT_STATUS.md includes a 'Last synced to Claude.ai' field. Updated when user pastes or uploads to Claude.ai.

Non-project items: Items that aren't software projects (content, operations, fitness goals) appear in the PDF under thematic headings but do not get progress bars or repo status.

6am automation: If the Mac is asleep at 6am, launchd will run the job when the Mac next wakes. The PDF overwrites the previous day's file (timestamped filename prevents conflicts).

Asset detection: When running sync_history.py --assets, the script scans Claude Code transcripts for file creation patterns matching known categories (auth, notifications, UI components, etc.).

Completed projects: Projects at 100% stay in the dashboard for reference but can be moved to an 'Archived' section in PROJECT_STATUS.md.

6. Data Requirements

Per-Project Data (stored in PROJECT_STATUS.md)

Field	Type	Example	Required
Project name	String	ParentPoint	Yes
Repository	String / URL	christreadaway/parentpoint	No
Category	Enum	School	Yes
Progress	Integer (0-100)	70	Yes
Status	String	Beta Live	Yes
Last worked date	Date	2026-02-01	Yes
Has GitHub repo	Boolean	Yes	Yes
Action needed	String	Create repo, deploy	If no repo
Progress log entries	Dated text blocks	See Section 3	Yes
Reusable assets	Table entries	See Section 10	No

Global Data (stored in generate_status_pdf.py)

The PDF generator contains a PROJECTS dictionary that serves as the master data source for the dashboard. This dictionary must be manually updated when projects are added, removed, or change status. Each entry includes: name, repo (if any), status string, progress integer, and category.

Asset Registry Data (stored in SHARED_ASSETS.md)

Field	Type	Example
Asset type	Enum	Authentication, Notifications, UI Components
File path	String	src/auth/googleAuth.js
Project	String	ParentPoint
Description	String	Google OAuth flow for mobile + web
Tags	Comma-separated	auth, google, oauth, mobile
Detected date	Date	2026-01-15

7. Integrations & Dependencies

System / Tool	Role	Integration Type	Notes
Claude Code	Primary input source	Local file writes	Writes PROJECT_STATUS.md automatically
Claude.ai	Strategy + chat interface	Upload / paste	Reads PDFs and pasted markdown
GitHub	Code hosting + repo tracking	Reference only	Repo URLs stored in status files
Python 3 + reportlab	PDF generation	Local script	Generates charts and tables
Python 3 + matplotlib	Chart rendering	Local script	Backup chart library
Mac launchd	Scheduling	System daemon	6am daily trigger
pbcopy (Mac)	Clipboard sync	CLI utility	Quick copy for paste into Claude.ai
Google Drive (optional)	Cloud backup	File sync	Optional: save PDFs to Drive folder

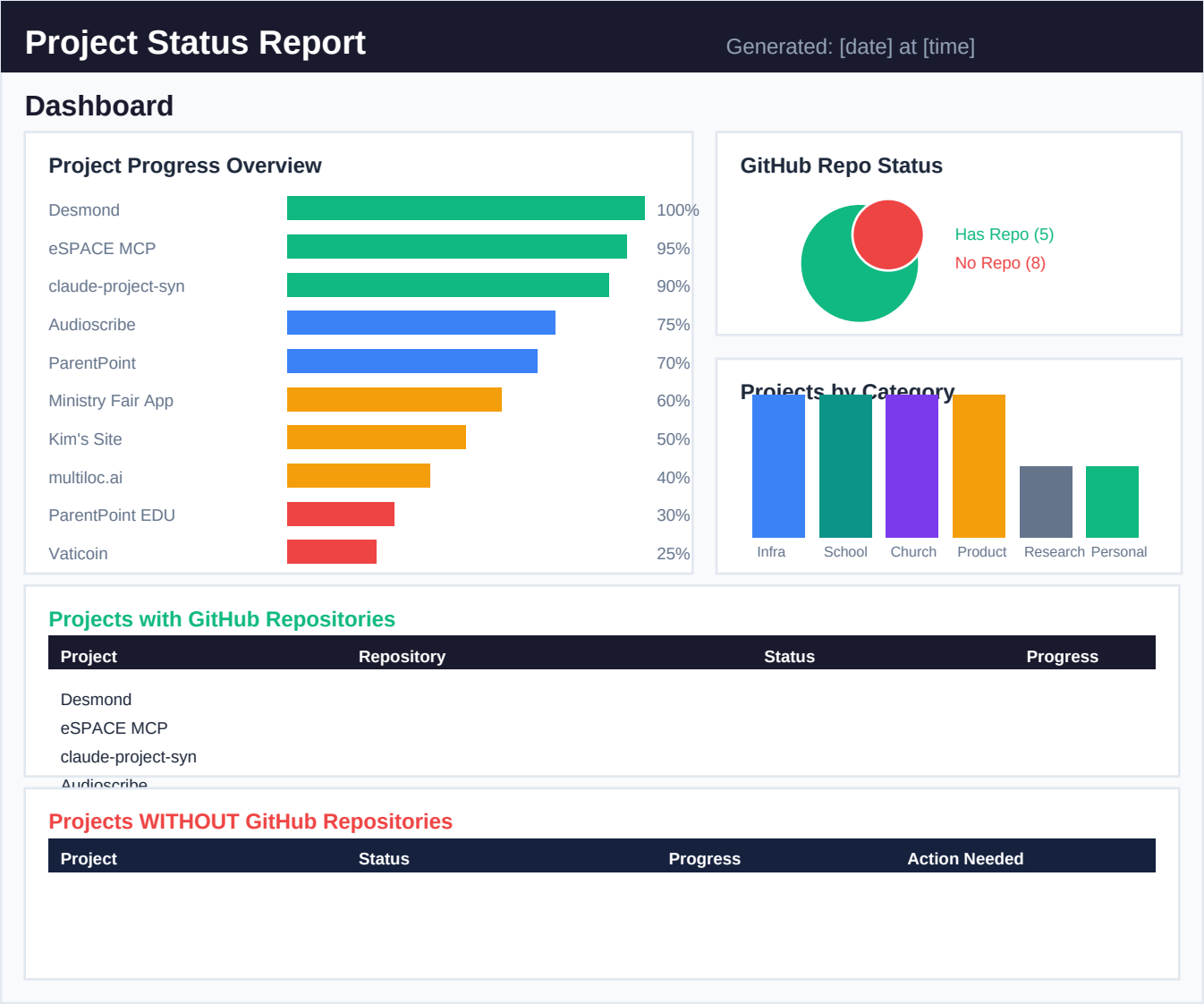
Runtime Dependencies

Package	Version	Purpose
Python	3.10+	Script runtime
reportlab	Latest	PDF generation with Platypus
matplotlib	Latest	Chart rendering (progress bars, pie, bar)
macOS	12+	launchd scheduling

8. Dashboard Visual Specification

This section defines the exact layout, chart types, and visual hierarchy for the daily PDF dashboard. This is the reference design. If generate_status_pdf.py is ever lost or rebuilt, recreate this layout exactly.

Page 1 Layout: Dashboard



Wireframe: Page 1 Dashboard Layout

Chart Specifications

Chart	Type	Data Source	Color Logic
Progress Overview	Horizontal bar chart	All projects sorted by % desc	Green > 75%, Blue 50-75%, Yellow 25-50%, Red < 25%
GitHub Repo Status	Pie chart	Count of has_repo vs no_repo	Green = has repo, Red = no repo

Projects by Category	Vertical bar chart	Count of projects per category	Unique color per category
----------------------	--------------------	--------------------------------	---------------------------

Page 2+: Overflow Tables & Thematic Items

If projects exceed the space on page 1, the 'Projects WITHOUT Repos' table and all Non-Project Items flow to page 2. Non-project items are grouped under thematic headers: Content & Thought Leadership, Church Operations, School Operations, and Research & Personal. Each item shows a brief status string (e.g., 'Complete', 'Active', 'Not Started').

The final line of the PDF reads: *If this tool saves you time: Venmo @ctreada*

9. Current Project Statuses

As of February 03, 2026

Projects with GitHub Repositories

Project	Repository	Status	Progress	Category
claude-project-sync	christreadaway/claude-project-sync	Built	90%	Infrastructure
ParentPoint	christreadaway/parentpoint	Beta Live	70%	School
ParentPoint EDU	Connected to ParentPoint repo	Ready to Build	30%	School
Ministry Fair App	christreadaway/ministryfair	Needs Testing	60%	Church
Desmond	Public repo	v1 Shipped	100%	Infrastructure
multiloc.ai / Polygraph	christreadaway/polygraph	Domain Bought	40%	Product

Projects WITHOUT GitHub Repositories

Project	Status	Progress	Action Needed
Audioscribe	v1 Working	75%	Push to GitHub after Patreon
eSPACE MCP	Complete	95%	Create repo, document
RenWeb MCP	Planned	10%	Scope complete, build next
MinistryPlatform MCP	Planned	10%	Scope complete, build next
Bluewave MCP	Planned	5%	Requires SQL approach
Vaticoin	Shelved	25%	Waiting on Vatican interest
Kim's Site	Deadline-driven	50%	WordPress, no repo needed

Non-Project Items (Thematic)

Content & Thought Leadership

Item	Status
Catholic MBA	GTM Built, Not Launched
Modernize Catholic	2 Posts Done
Book Project	Not Started

Church Operations

Item	Status
Emergency Procedures	Complete
Facilities Onboarding	Complete
Sacramental Records	Concept Stage

School Operations

Item	Status
The Human Code Class	Teaching Active

Research & Personal

Item	Status
Randy	TBD
Ralph Wiggums	TBD
HEB Scraper	Blocked
2026 Fitness Goals	Active

10. Shared Assets Registry

These are reusable components identified across projects. When starting a new project, check this registry before building from scratch. Run `python3 sync_history.py --assets` to refresh.

Infrastructure & Tooling Assets

Asset	Project	Description	Tags
generate_status_pdf.py	claude-project-sync	PDF report with charts (reportlab + matplotlib)	pdf, charts, reportlab
init_project_status.py	claude-project-sync	CLI tool to initialize PROJECT_STATUS file templates, init	cli, templates, init
sync_history.py	claude-project-sync	Imports Claude Code transcripts + builds asset registry	sync, history, assets
daily_status_report.sh	claude-project-sync	Shell wrapper for launchd automation	shell, automation, cron
Global CLAUDE.md	claude-project-sync	Template for Claude Code instructions + config, Claude	config, claude

MCP Server Assets

Asset	Project	Description	Tags
eSPACE MCP Server	eSPACE MCP	Complete MCP server for eSPACE facility management facilities	mcp, espace, facilities
RenWeb MCP (scoped)	RenWeb MCP	Scope document ready for SIS integration	mcp, renweb, school
MinistryPlatform MCP (scoped)	MinistryPlatform MCP	Scope document ready for ChMS integration	mcp, ministryplatform, church

Product & Application Assets

Asset	Project	Description	Tags
ParentPoint mobile app	ParentPoint	React Native app for school parent communication	reactnative, mobile, school
Ministry Fair web app	Ministry Fair App	Web app for church ministry volunteer signups	web, church, volunteer
Desmond CLI	Desmond	Shipped v1 command-line tool	cli, shipped, public
Audioscribe v1	Audioscribe	Working audio transcription tool	audio, transcription, ai
Polygraph / multiloc.ai	multiloc.ai	Multi-language AI translation tool	translation, ai, multilingual

11. Out of Scope

The following are explicitly NOT part of Claude Project Sync at this time:

Google Docs write integration. Both Claude Code and Claude.ai lack write access to Google Docs. This was the v1-v3 approach and was abandoned after testing proved it unfeasible.

Real-time sync. There is no live connection between Claude Code and Claude.ai. The sync is file-based and requires a manual upload/paste step (~5 seconds).

Multi-user / team features. The system is designed for a single user managing their own projects. Team sync is a future enhancement.

Database or cloud storage. All data lives in local files. No database, no cloud API, no server.

Automated Claude.ai uploads. There is no API to programmatically upload files to Claude.ai. The user must do this manually.

Windows / Linux scheduling. The automation is Mac-only (launchd). Windows Task Scheduler and Linux cron equivalents are not yet built.

Mobile interface. No mobile app or responsive web dashboard. The output is a static PDF.

Git-based auto-sync. The system does not watch git commits or automatically detect when repos change.

12. Open Design Questions

Q: Should the PDF generator pull from PROJECT_STATUS.md files directly?

Currently, project data is hardcoded in generate_status_pdf.py. An alternative is to have the script scan all PROJECT_STATUS.md files and build the dashboard dynamically. This would eliminate the need to manually update the Python dictionary.

Q: How should progress percentages be calculated?

Currently set manually. Could be derived from git activity, issue counts, or milestone completion. Manual feels more accurate for non-code projects.

Q: Should there be a web dashboard instead of (or in addition to) the PDF?

A static HTML page could be generated alongside the PDF, offering interactive filtering. But the PDF's simplicity and uploadability to Claude.ai is a strength.

Q: How do we handle project archival?

Desmond is at 100%. Should completed projects age off the dashboard after N days? Or always stay visible?

Q: Should SHARED_ASSETS.md auto-generate or be manually curated?

The sync_history.py --assets flag does automated detection, but it may miss context. A hybrid approach (auto-detect + manual curation) may work best.

Q: What happens when the number of projects exceeds 15-20?

The current dashboard layout works well for ~13 projects. At 20+ the progress bar chart may need pagination or a different visualization.

Q: Should non-project items get progress tracking?

Items like 'Book Project: Not Started' could benefit from progress bars, but they don't map cleanly to percentage completion.

13. Success Criteria

How We Know It's Working

Context continuity: When switching from Claude Code to Claude.ai (or vice versa), fewer than 2 messages are needed to get Claude up to speed. Previously: 5-10 messages of re-explanation.

Daily PDF generation: The 6am PDF generates successfully every day without manual intervention. Validated by checking the file timestamp.

Project coverage: All active projects (currently 13) appear in the dashboard with accurate progress and status.

Sync time under 10 seconds: The manual step (copy/paste or upload) takes less than 10 seconds per sync event.

Asset reuse: At least once per month, the shared asset registry prevents rebuilding a component that already exists in another project.

Zero data loss: No project progress or decisions are lost when switching between Claude Code and Claude.ai sessions.

Developer handoff: A new developer (or Claude in a fresh session) can read this spec and rebuild the system without access to the original conversations.

Measurable Targets

Metric	Target	How to Measure
Context re-establishment	< 2 messages	Count messages before Claude is "caught up"
PDF generation reliability	95%+ daily	Check file timestamps over 30 days
Manual sync friction	< 10 seconds	Time the copy-paste or upload step
Project accuracy	100% of active projects shown	Compare dashboard to known project list
Asset registry hits	1+ per month	Track times registry prevented rebuild

14. Future Enhancements

Auto-prompt at session end. Claude Code automatically asks 'update status?' at natural stopping points, reducing the chance of forgetting to sync.

Dynamic PDF from status files. `generate_status_pdf.py` scans all `PROJECT_STATUS.md` files instead of using a hardcoded dictionary. Adding a project means creating a status file, not editing Python.

Smarter asset detection. Use pattern matching and Claude Code transcript analysis to automatically identify and categorize reusable components.

Dependency mapping. Track which projects depend on shared assets, so changes in one project flag potential impacts on others.

Version history / changelog. Maintain a running changelog of project status over time, enabling trend analysis (is this project accelerating or stalling?).

Team sync. Optional sharing of status docs across team members for projects with multiple contributors.

MCP integration. Native Model Context Protocol server for tighter, more seamless integration between Claude Code and Claude.ai.

HTML dashboard option. Generate a static HTML page alongside the PDF for interactive filtering and drill-down.

Cross-platform scheduling. Windows Task Scheduler and Linux cron equivalents for non-Mac users.

End of Specification

This document was generated from conversation history between Chris Treadaway and Claude. It is designed so that a developer or Claude in a future session can build from it without needing the original conversations.

If this tool saves you time: Venmo @ctreada